



DSA & C++

Sorting Algorithms

shorturl.at/ipGZ9

22

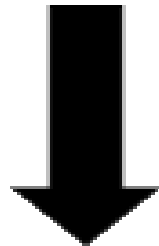
13

5

7

11

Unsorted array



Sorted array

5

7

11

13

22

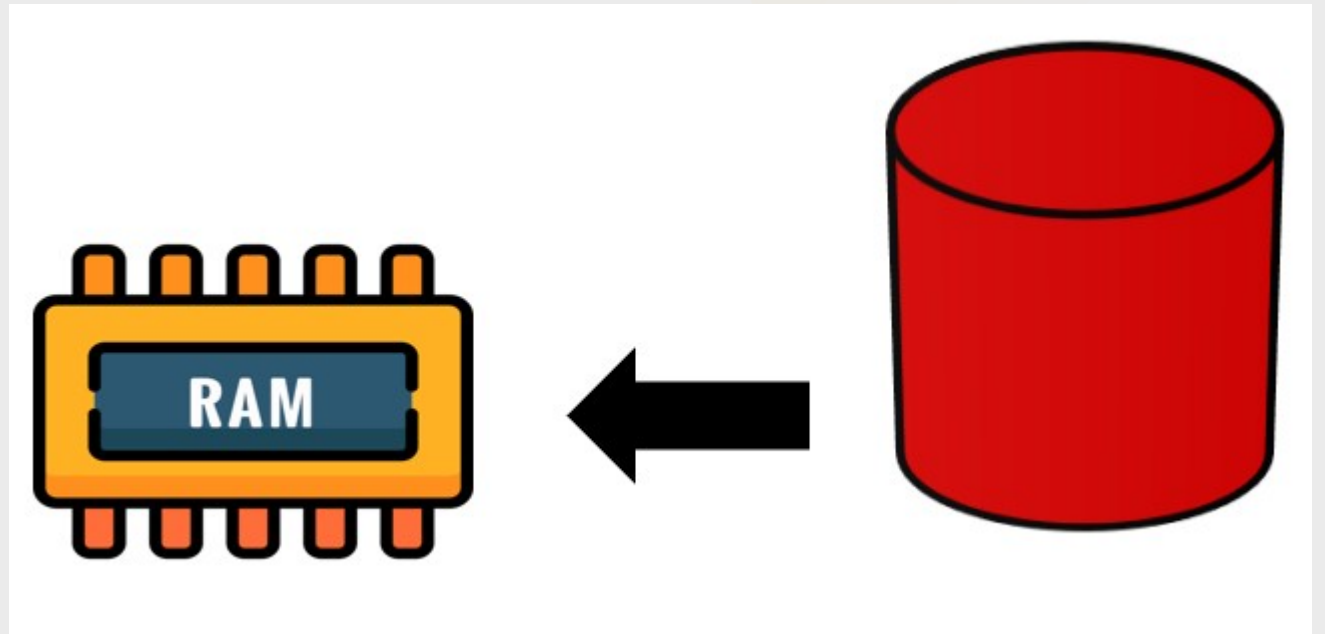
Introduction

- ▶ **Sorting** refers to simply the process of arranging the data elements of an array in a specified order, that is, either in ascending or descending order.
- ▶ For example, it will be practically impossible for us to find a name in the telephone directory if the names in it are not in alphabetical order. However, the same can be true for dictionaries, book indexes, bank accounts, and so on.
- ▶ Hence, the convenience of having sorted data is unquestionable. Retrieval of information becomes much easier when the data is stored in some specified order. Therefore, sorting is a very important application in computer science. Arranging things in a sorted manner makes it easier to analyze and search for a particular element among the collection of elements.
- ▶ **A sorting algorithm** can be defined as an algorithm that puts the data elements of an array/ list in a certain order, that is, either numerical order or any predefined order. There are many sorting algorithms that are available and are widely used according to the different environments required by the different sorting methods.

Categories of Sorting

- ▶ **The two basic categories of sorting methods are:**

- ▶ **Internal Sorting** – It refers to the sorting of the data elements stored in the computer's main memory.
 - ▶ Example: Bubble, selection and insertion sort
- ▶ **External Sorting** – It refers to the sorting of the data elements stored in the files. It is applied when the amount of data is large and cannot be stored in the main memory.
 - ▶ Example: Merge Sort



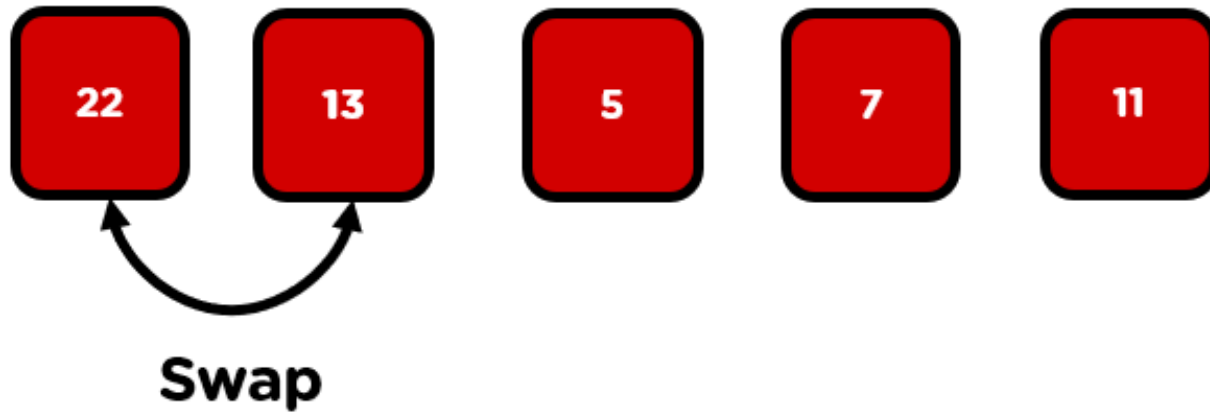
Types of Sorting

- ▶ There are various types of sorting techniques in C++. We will be learning the most popular ones in this article.
 - ▶ Bubble sort
 - ▶ Insertion sort
 - ▶ Selection sort
 - ▶ Quick sort
 - ▶ Merge Sort

*See also more sorting
algorithms
Counting
Bucket
Radix
heap*

Bubble Sort

- ▶ **Bubble Sort** is the very simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. In this sorting technique, we begin by comparing the first two elements of the array and checking if the first element is greater than the second element; if it is, we will swap those elements and move forward to the next element



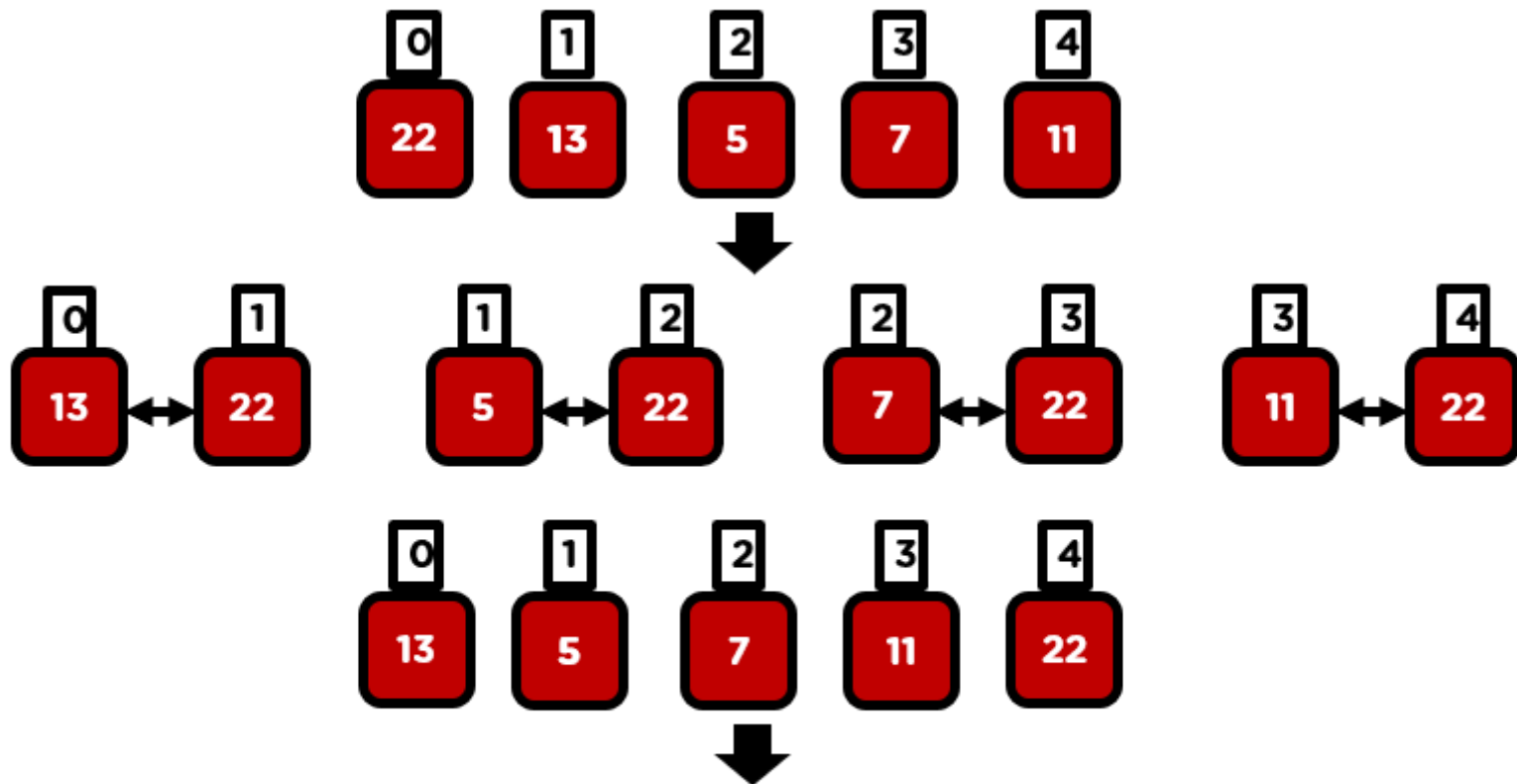
If the first element is not greater than the second, then we don't need to swap it. And this process will keep on repeating till the end of the array.

Bubble sort(next...)

- ▶ With Bubble Sort , two successive elements are compared with each other, and – if the left element is larger than the right one – they are swapped.
- ▶ These comparison and swap operations are performed from left to right across all elements. Therefore, after the first pass, the largest element is positioned on the far right. Or better: *at the latest* after the first pass – it may have arrived there before.
- ▶ You repeat this process until the array is sorted.
- ▶ Let's use an example for the better understanding, In the following visualizations, I show how to sort the array [22,13,5,7,11] with Bubble Sort:

Bubble sort(next..)

- ▶ We divide the array into a left, unsorted – and a right sorted part. The right part is empty at the beginning:



Now again the comparison will take place till the array becomes sorted. For n number of elements the comparisons are $(n-1)$.

Iteration 1:

We compare the first two elements, the 22 and the 13, and since 13 is smaller, we swap the elements:

Now we compare the second with the third element, i.e., the 22 with the 5. These are also in the wrong order and are, therefore, swapped. We compare the third with the fourth element, i.e., the 22 with the 7, they are also swapped.

Example1: Visual Interpretation

Input: arr[] = {22,13,5,7,11}

Comparison 1:

- Bubble sort starts with very first two elements, comparing them to check which one is greater.
 - {**22,13**,5,7,11} → {**13,22**,5,7,11} Here, algorithm compares the first two elements, and swaps since $22 > 13$.
 - {13,**22,5**,7,11} → {13,**5,22**,7,11} , Swap since $22 > 5$
 - {13,5,**22,7**,11} → {13,5,**7,22**,11}, Swap since $22 > 7$
 - {13,5,7,**22,11**} → {13,5,7,**11, 22**}, Swap since $22 > 11$.

Comparison 2:

- Bubble sort in the second iteration will look like this:
 - {**13,5**,7,11, 22} → {**5,13**, 7,11, 22} Here, algorithm compares the first two elements, and swaps since $13 > 5$.
 - {5,**13, 7**,11, 22} → {5,**7,13**,11, 22} , Swap since $13 > 7$
 - {5,7,**13,11**, 22} → {5,7,**11,13**, 22} Swap since $13 > 11$
 - {5,7,11,**13, 22**} → {5,7,11,**13, 22**} , No Swap since $13 < 22$.

Comparison 3:

- Currently, the array is sorted, but our algorithm does not know if it is completed.
- The algorithm needs one **whole** pass without **any** swap to know it is sorted.
 - {**5,7**,11,13, 22} → {**5,7**,11,13, 22}
 - {5,**7,11**,13, 22} → {5,**7,11**,13, 22}
 - {5,7,**11,13**, 22} → {5,7,**11,13**, 22}
 - {5,7,11,**13, 22**} → {5,7,11,**13, 22**}

Example 2: Steps and Number of Comparison

// n=4, comparison=n-1

```
int arr[4]={12,55,5,18};
```

The number of comparison is always n-1

- If **arr[j]** is greater than **arr[j+1]** then swap these adjacent elements, else move on

Unsorted array			
12	55	5	18
Comparison=0			
j=0			
12	55	5	18
j=1			
12	5	55	18
j=2			
12	5	18	55
=====			
Comparison=1			
j=0			
5	12	18	55
j=1			
5	12	18	55
=====			
Comparison=2			
j=0			
5	12	18	55
=====			
Sorted array			
5	12	18	55

Example 2: Steps and Number of Comparison

```
#include<iostream>
using namespace std;
int main(){
int iteration,j,temp,n=9;
int num[n]={1,2,3,4,5,6,7,8,9};
cout<<" The unsorted array"<<endl;
for(int i=0;i<n;i++){
cout<<num[i]<<"\t";
}
cout<<endl;
for(iteration=1;iteration<n;iteration++){
cout<<"Iteration="<<iteration<<endl;
int check=0;
for (j=0;j<n-1;j++){
if(num[j+1]<num[j]){
temp=num[j];
num[j]=num[j+1];
num[j+1]=temp;
check=1;
}
```

```
cout<<endl;
cout<<"J="<<j<<endl;
for(int k=0;k<n;k++){
cout<<num[k]<<"\t";
}
cout<<endl;
}
if(check==0){
break;
}
}
cout<<"The sorted array is "<<endl;
for(int i=0;i<n;i++){
cout<<num[i]<<"\t";
}
cout<<endl;

return 0;
}
```

Example 3: Visual illustration

```
int numbers[7]={5,3,1,9,8,2,4,7};
```

The number of comparison is always $n-1$

- If **arr[j]** is greater than **arr[j+1]** then swap these adjacent elements, else move on

i = 0	j	0	1	2	3	4	5	6	7
	0	5	3	1	9	8	2	4	7
	1	3	5	1	9	8	2	4	7
	2	3	1	5	9	8	2	4	7
	3	3	1	5	9	8	2	4	7
	4	3	1	5	8	9	2	4	7
	5	3	1	5	8	2	9	4	7
	6	3	1	5	8	2	4	9	7
i = 1	0	3	1	5	8	2	4	7	9
	1	1	3	5	8	2	4	7	
	2	1	3	5	8	2	4	7	
	3	1	3	5	8	2	4	7	
	4	1	3	5	2	8	4	7	
	5	1	3	5	2	4	8	7	
i = 2	0	1	3	5	2	4	7	8	
	1	1	3	5	2	4	7		
	2	1	3	5	2	4	7		
	3	1	3	2	5	4	7		
	4	1	3	2	4	5	7		
i = 3	0	1	3	2	4	5	7		
	1	1	3	2	4	5			
	2	1	2	3	4	5			
	3	1	2	3	4	5			
i = 4	0	1	2	3	4	5			
	1	1	2	3	4				
	2	1	2	3	4				
i = 5	0	1	2	3	4				
	1	1	2	3					
i = 6	0	1	2	3					
		1	2						

C++ Program

```
#include<iostream>

using namespace std;

int main(){

int i,count,j,temp,n=8;

int arr[8]={12,3,1,5,18,10,7,35};

cout<<"Unsorted array"<<endl;

for(i=0;i<n;i++){

cout<< arr[i]<<"\t";

}

cout<<endl;

cout<<endl;
```

```
for(count=1;count<=n-1;count++){

for(j=0;j<=n-2;j++)

{

if(arr[j+1]<arr[j]){

temp=arr[j];

arr[j]=arr[j+1];

arr[j+1]=temp;

}

}

cout<<"Sorted array"<<endl;

for(i=0;i<n;i++){

cout<< arr[i]<<"\t";

}

cout<<endl;

return 0;

}
```

C++ Program with Swap function

```
#include<iostream>

using namespace std;

int main(){

int i,count,j,temp,n=8;

int arr[8]={12,3,1,5,18,10,7,35};

cout<<"Unsorted array"<<endl;

for(i=0;i<n;i++){

cout<< arr[i]<<"\t";

}

cout<<endl;

cout<<endl;
```

```
for(count=1;count<=n-1;count++){

for(j=0;j<=n-2;j++)

{

if(arr[j+1]<arr[j]){

swap(arr[j],arr[j+1]);

}

}

}

cout<<"Sorted array"<<endl;

for(i=0;i<n;i++){

cout<< arr[i]<<"\t";

}

cout<<endl;

return 0;

}
```

Introduction to Time Complexity

In the Bubble sort:

- ▶ The inner loop will executes $n-1$ times
- ▶ The outer loop will execute $n-1$ times
- ▶ The number of steps= $(n-1)*(n-1);$

$$=n^2+n+c$$

$$\cong n^2$$

- ▶ The number of steps is called the Time complexity(How many steps the program took to execute?)
- ▶ The time complexity is written with notation O
- ▶ The time complexity of Bubble sort is $O(n^2)$

Note: The time complexity assists to make a choice of the best algorithm with a least amount of time

- As we can see in the above example, we have an unsorted array containing elements
- 12, 3, 1, 5, 18, 10, 7 and 35.
- Run a nested for loop to traverse the input array using two variables **i** and **j**, such that $0 \leq i < n-1$ and $0 \leq j < n-i-1$
- If **arr[j]** is greater than **arr[j+1]** then swap these adjacent elements, else move on
- j must be not greater than n-2 to avoid segmentation fault
- To sort n numbers, perform a sorting of n-1 numbers.
- In the first iteration, the first largest number reaches its final position, in the second iteration the second largest reaches its final position, and the process continue.
- Print the sorted array
- The output will look like this:

```
Unsorted array :  
12      3      1      5      18     10     7      35  
Sorted Elements ...  
1       3       5       7      10     12     18     35
```




Exercise: Indicate the steps and comparison of the bubble sort of the following array sorted in reverse order

```
int arr[4]={55,18,12,5};
```

Optimisation of Bubble sort

Consider the following arrays A and B:

8	9	12	5	20
---	---	----	---	----

5	8	9	12	20
---	---	---	----	----

The time complexity of A is $O(n^2)$ and the time complexity of B is $O(n^2)$

If the array is in the correct order, the same time complexity will be used. It can be optimized by stopping the algorithm if the inner loop didn't cause any swap. This means the Sorted array will take n steps in the bubble sort $O(n)$ which is the best time complexity.

Example: Bubble sort optimised

```
#include<iostream>

using namespace std;

int main(){

int i,count,j,temp,n=8;

int arr[8]={12,3,1,5,18,10,7,35};

cout<<"Unsorted array"<<endl;

for(i=0;i<n;i++){

cout<< arr[i]<<"\t";

}

cout<<endl;

cout<<endl;
```

```
for(count=1;count<=n-1;count++){
    int check=0;
    for(j=0;j<=n-2;j++)
    {
        if(arr[j+1]<arr[j]){
            swap(arr[j],arr[j+1]);
            check=1;
        }
    }
    if(check==0){
        break;//The array is sorted
    }
}

cout<<"Sorted array"<<endl;
for(i=0;i<n;i++){
    cout<< arr[i]<<"\t";
}

cout<<endl;
return 0;
}
```

2. Insertion Sort

- ▶ Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.

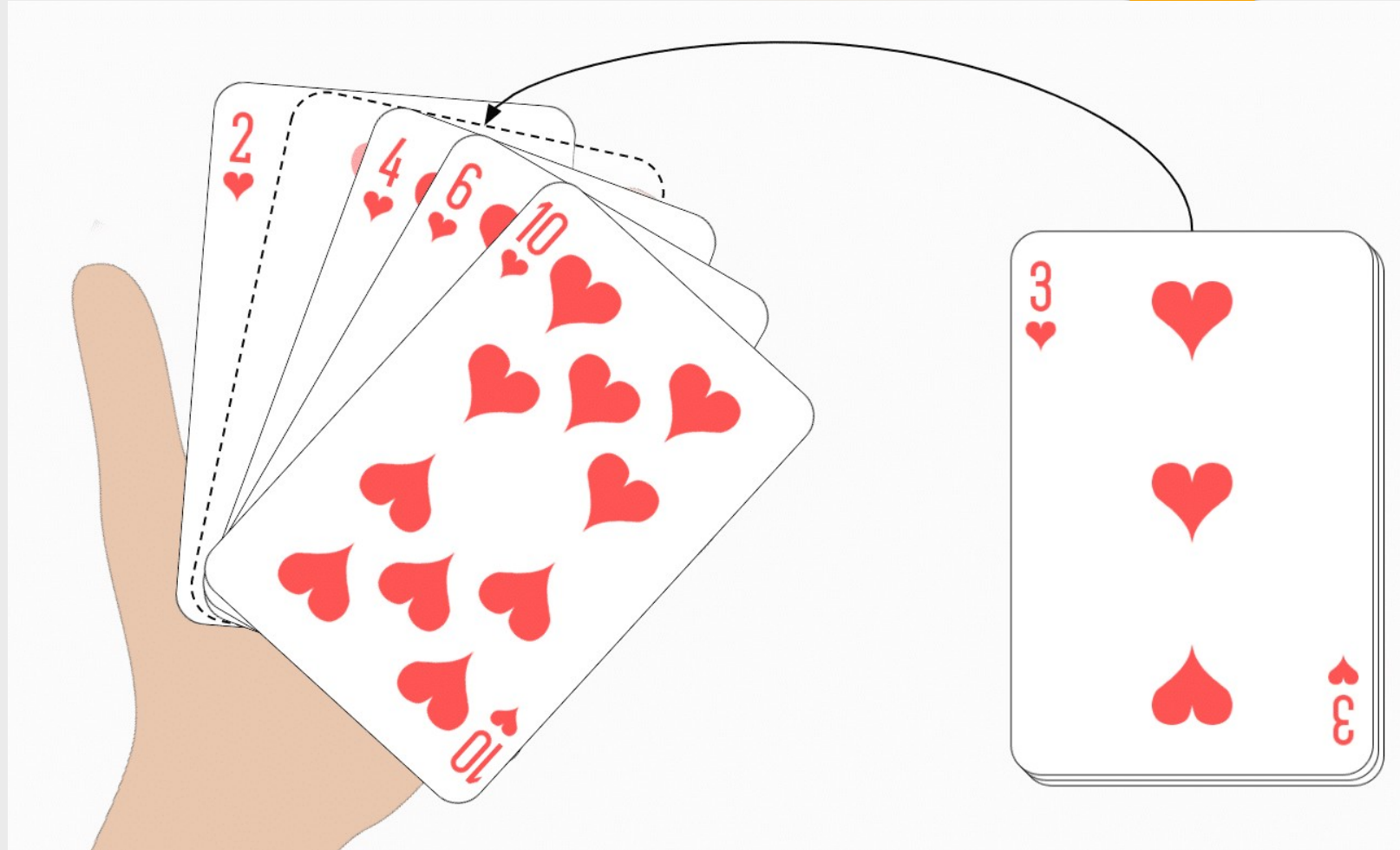
Algorithm

- ▶ To sort an array of size n in ascending order:
- ▶ Iterate from $\text{arr}[1]$ to $\text{arr}[n-1]$ over the array.
- ▶ Compare the current element (key) to its predecessor. It starts by comparing the second element with the first element. If the second element is smaller than the first, then we will swap it.
- ▶ If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

Let us go ahead with a playing card example.

Imagine being handed one card at a time. You take the first card in your hand. Then you sort the second card to the left or right of it. The third card is placed to the left, in between or to the right, depending on its size. And also, all the following cards are placed in the right position.

Have you ever sorted cards this way before?



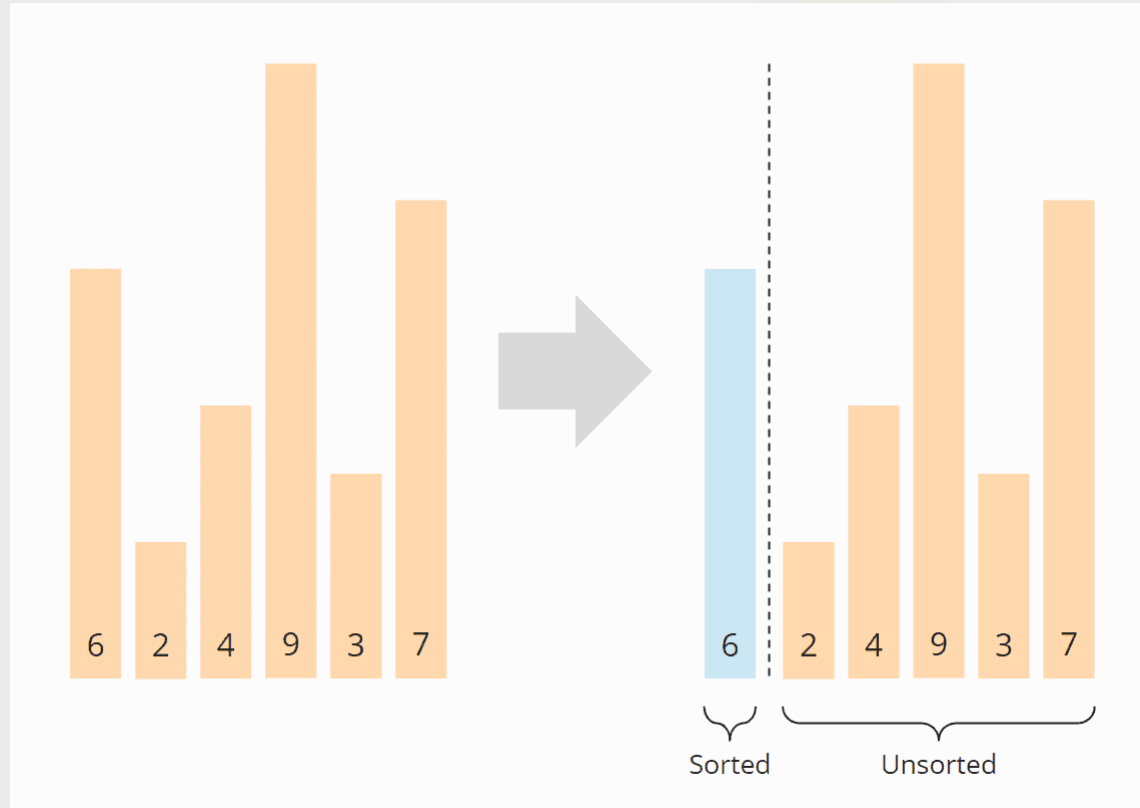
If so, then you have intuitively used "**Insertion Sort**".

Let's move from the card example to the computer algorithm. Let us assume we have an array with the elements [6, 2, 4, 9, 3, 7]. This array should be sorted with Insertion Sort in ascending order.

Step 1

First, we divide the array into a left, sorted part, and a right, unsorted part.

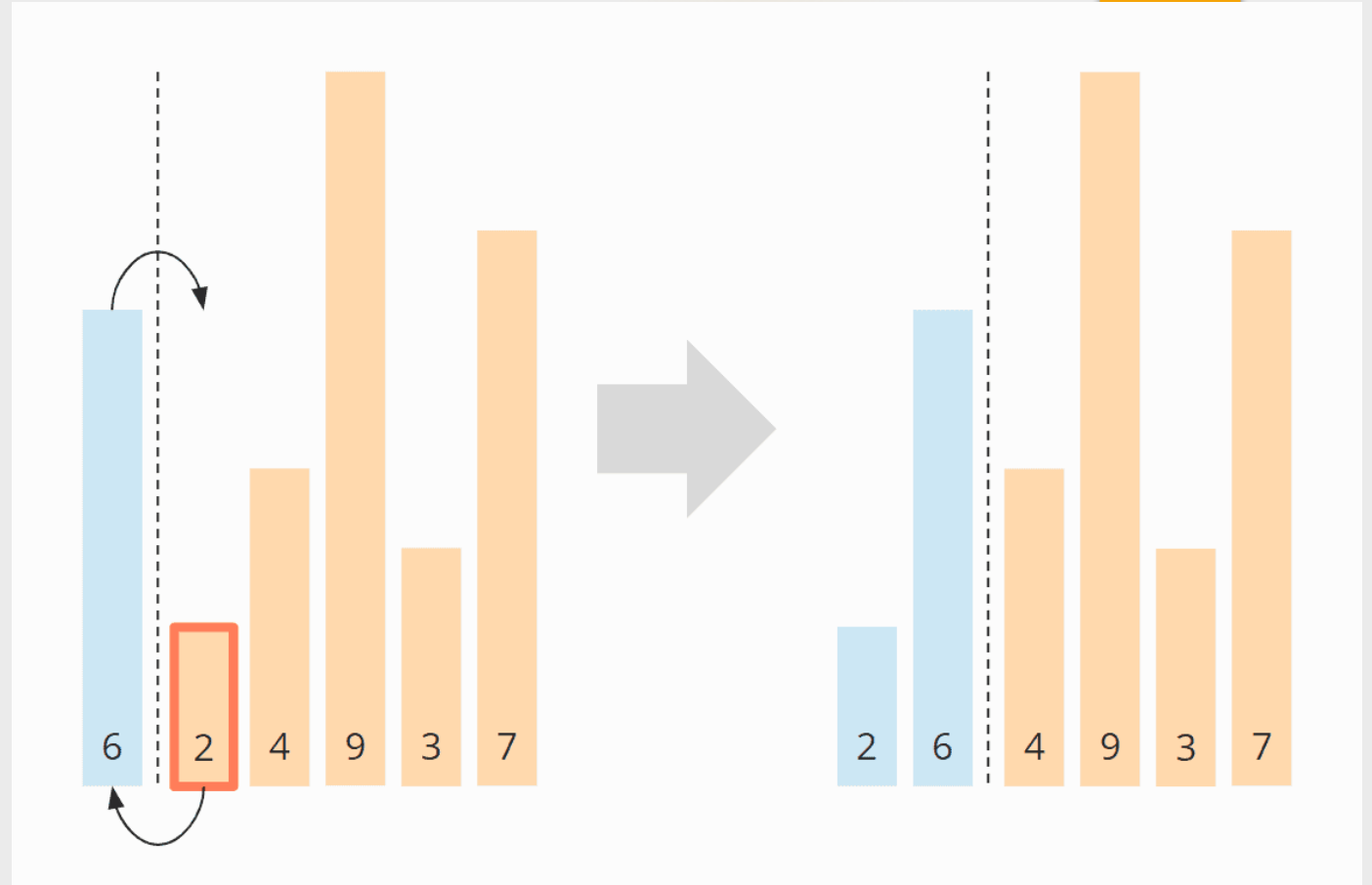
The sorted part already contains the first element at the beginning, because an array with a single element can always be considered sorted.



Step 2

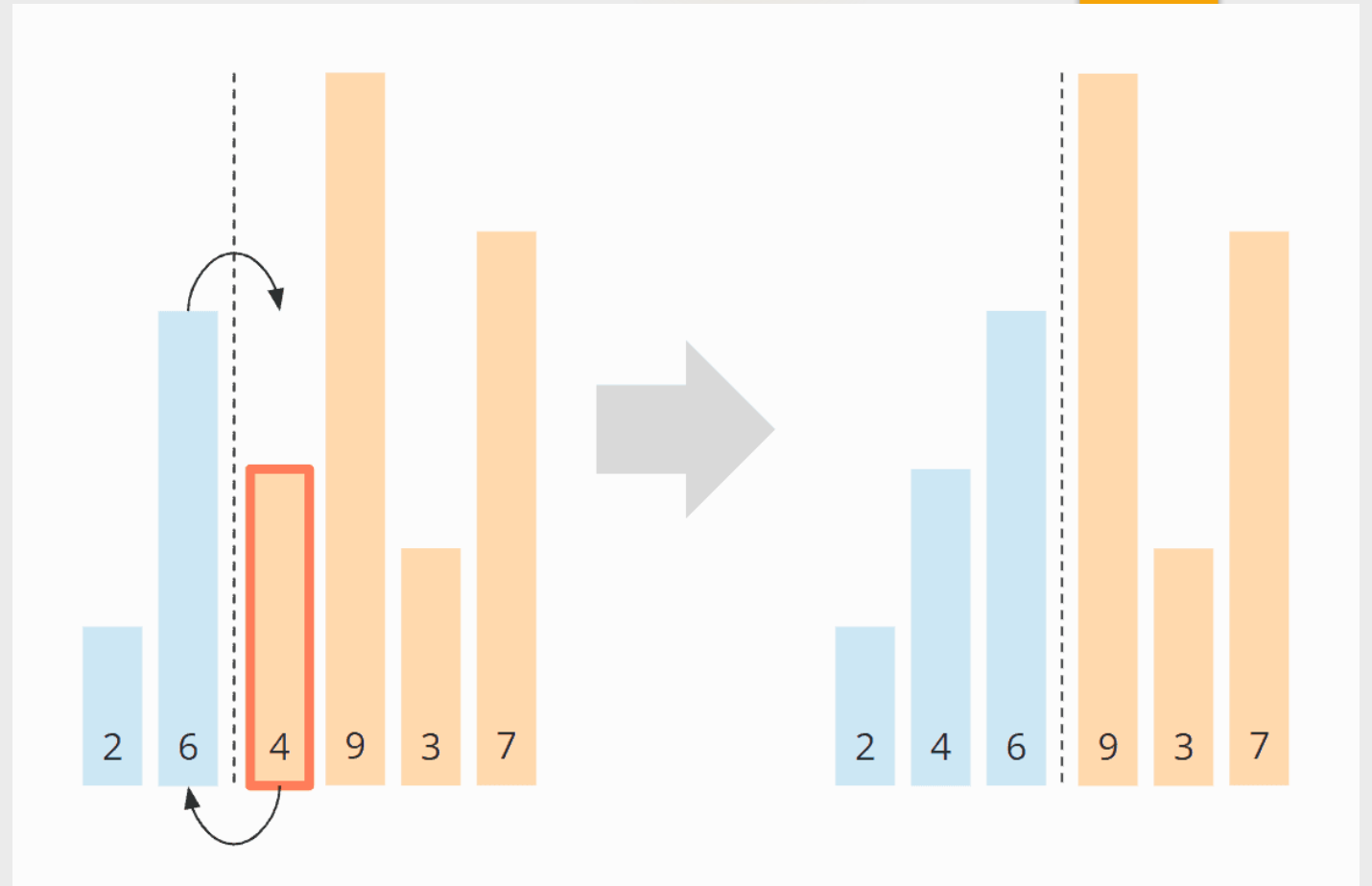
Then we look at the first element of the *unsorted* area and check where, in the sorted area, it needs to be inserted by comparing it with its left neighbor.

In the example, the 2 is smaller than the 6, so it belongs to its left. In order to make room, we move the 6 one position to the right and then place the 2 on the empty field. Then we move the border between the sorted and unsorted area one step to the right:



Step 3

We look again at the first element of the unsorted area, 4. It is smaller than 6, but not smaller than the 2 and, therefore, belongs between 2 and 6. So we move the 6 again one position to the right and place the 4 on the vacant field:



Step 4

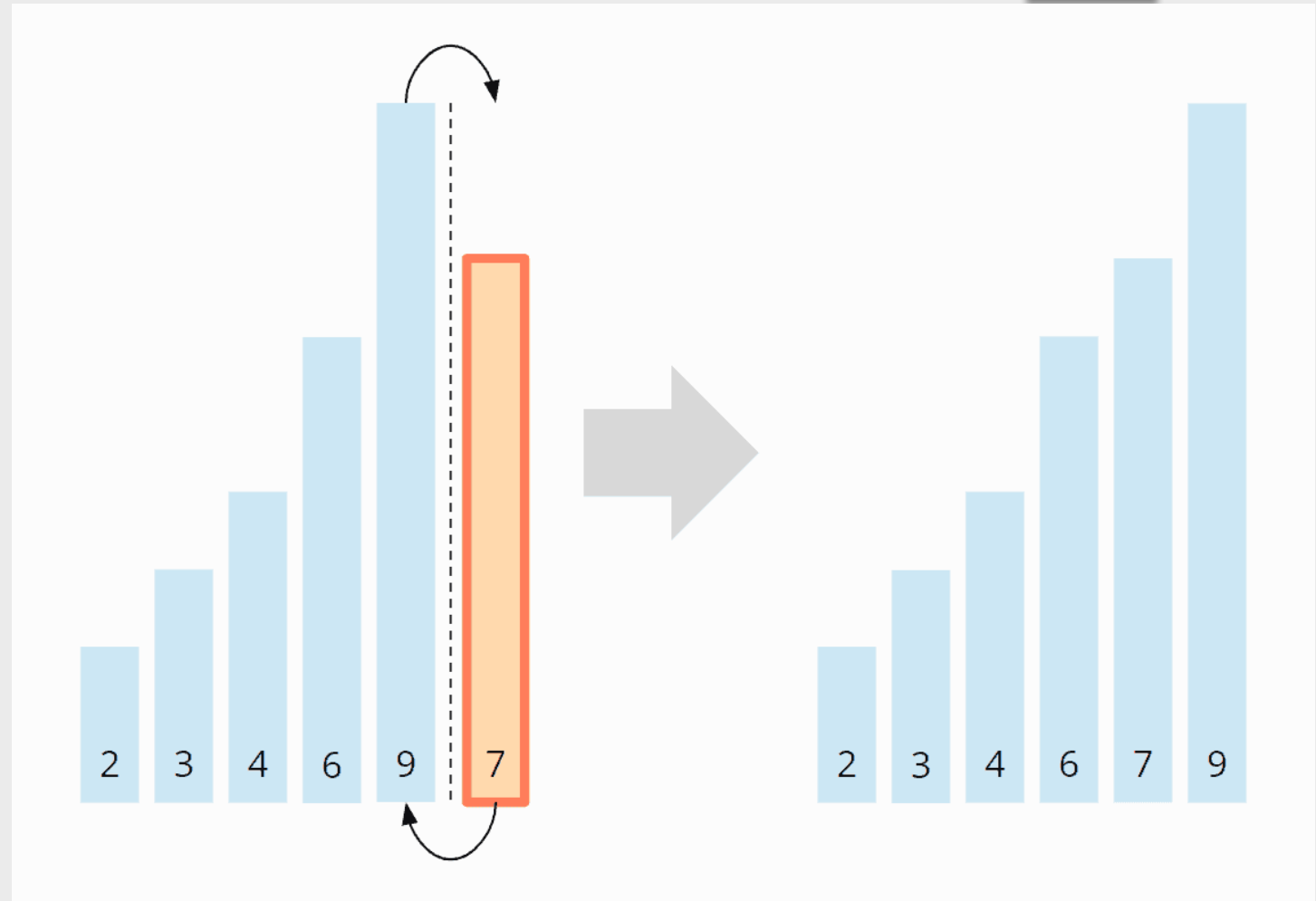
The next element to be sorted is the 9, which is larger than its left neighbor 6, and thus larger than all elements in the sorted area. Therefore, it is already in the correct position, so we do not need to shift any element in this step.

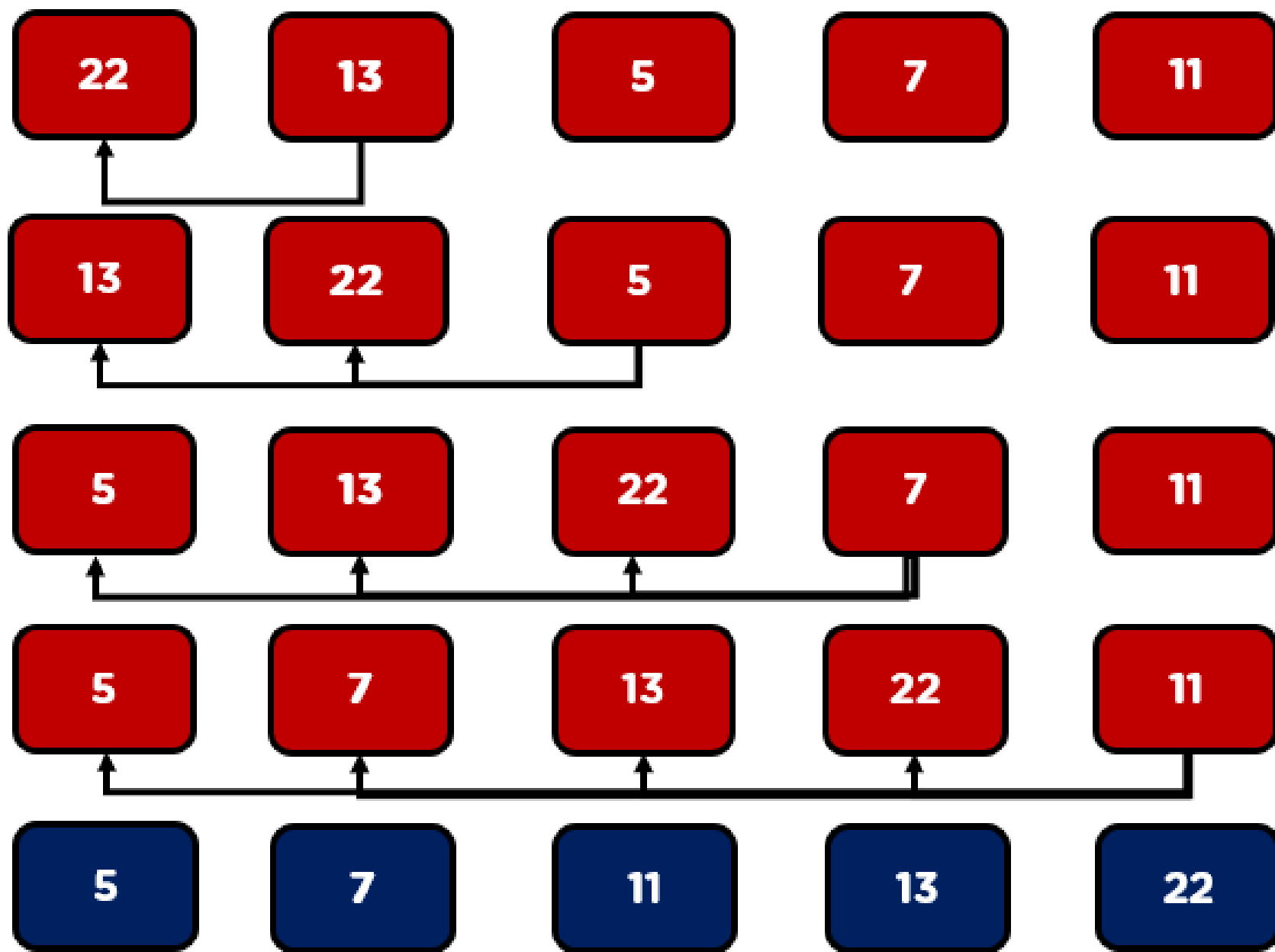
Step 5

The next element is the 3, which is smaller than 9, 6, and 4, but greater than the 2. So we move the 9, 6, and 4 one position to the right and then put 3 where the 4 was

Step 6

That leaves the 7 – it is smaller than the 9, but larger than the 6, so we move the 9 one field to the right and place the 7 on the vacant position:





- ▶ It starts by comparing the second element with the first element. If the second element is smaller than the first, then we will swap it.
- ▶ After that, we will compare the third element with all the elements that are before it. Similarly, it goes for the fourth element and so on. Once all the comparisons are made, the elements become sorted.

C++ program

► // C++ program for insertion sort
#include <bits/stdc++.h>
using namespace std;

/* Function to sort an array using insertion sort*/
void insertionSort(int arr[], int n)
{
 int i, key, j;
 for (i = 1; i < n; i++)
 {
 key = arr[i];
 j = i - 1;

 /* Move elements of arr[0..i-1], that
 are greater than temp, to one position ahead of their
 current position */
 while (j >= 0 && arr[j] > key)
 {
 arr[j + 1] = arr[j];
 j = j - 1;
 }
 arr[j + 1] = key;
 }
}



// A function to print an array of size n
void printArray(int arr[], int n)
{
 int i;
 for (i = 0; i < n; i++)
 cout << arr[i] << " ";
 cout << endl;
}

/* Driver code */
int main()
{
 int arr[] = { 12, 11, 13, 5, 6 };
 int n = **sizeof**(arr) / **sizeof**(arr[0]);

 insertionSort(arr, n);
 printArray(arr, n);

 return 0;

Insertion sort complexity

The insertion sort will take the time complexity of $O(n^2)$. The outer loop is taking $n-1$ steps and inner loop $n-2$ steps.

$$(n-1)*(n-2)=n^2+kn+c$$
$$\simeq n^2$$

- ▶ A for loop is used, which is iterating from 1 to num-1; the loop starts from 1 because the comparison will start from index 1.
- ▶ Inside the for loop, the array element is copied to the **key** variable, and j is set i-1 because we have to compare the element with its previous element(s).
- ▶ Inside the while loop, there's a condition if the **key** is less than arr[j], that means if **key** which is storing the index 1 element is less than arr[j], which is index 0 element, then we will put arr[j] into its correct position, i.e., arr[j+1].
- ▶ j=j-1 is used for decrementing the loop, as per the condition j must be greater than 0.

Exercise

1. Given the following array of 5 elements

- ▶ **int** arr[] = { 12, 11, 13, 5, 6 };
- ▶ Write a program to return the minimum and maximum number in the array

Selection Sort

Selection sort is a sorting technique that works by **finding the smallest value in the array and placing it in the first position.**

After that, it was then .

Thus, the selection sort works by finding the smallest unsorted element remaining in the entire array and then swapping it with the element in the next position to be filled.

It is a very simple technique, and it is also easier to implement than other sorting techniques. Selection sort is used for sorting files with large records.

Let's take the same example as in the selection sort of playing cards.

The algorithm maintains two subarrays in a given array.

1. The subarray is already sorted.
2. The remaining subarray is unsorted.

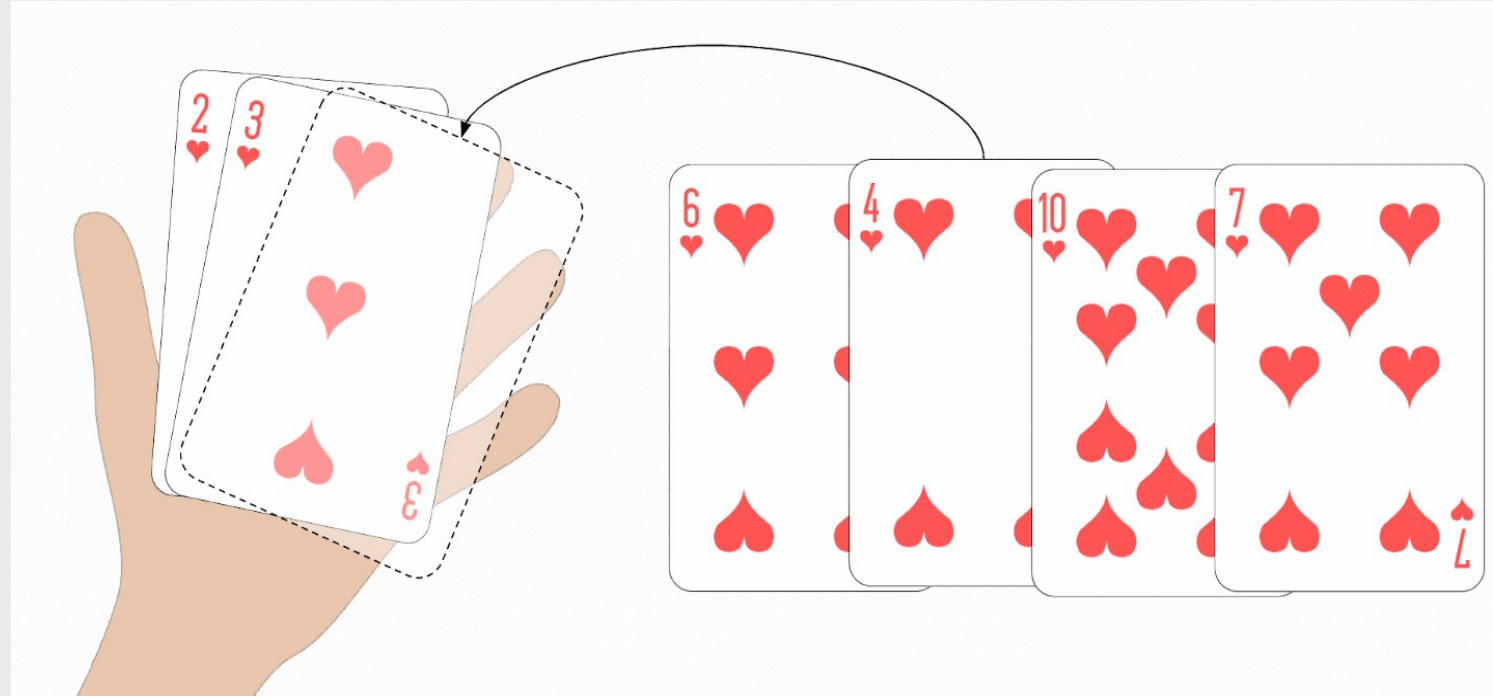
In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.

selection sort algorithm(next)

First, you lay all your cards face-up on the table in front of you. You look for the smallest card and take it to the left of your hand. Then you look for the next smaller card and place it to the right of the smallest card, and so on until you finally pick up the largest card to the far right.

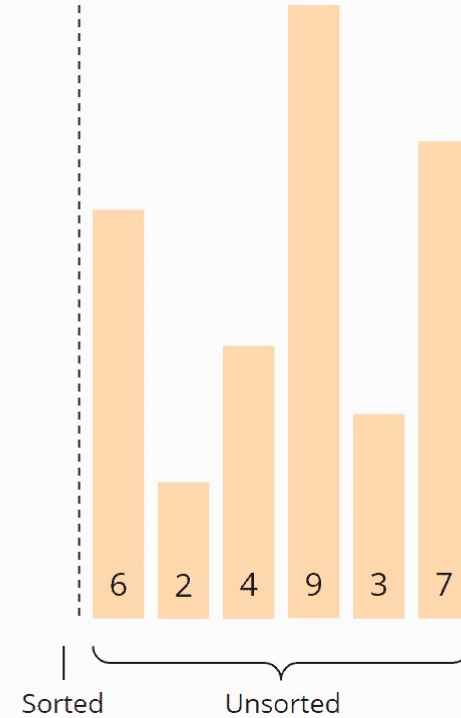
Selection Sort Algorithm

The algorithm can be explained most simply by an example. In the following steps, I show how to sort the array [6, 2, 4, 9, 3, 7] with Selection Sort:



Step1

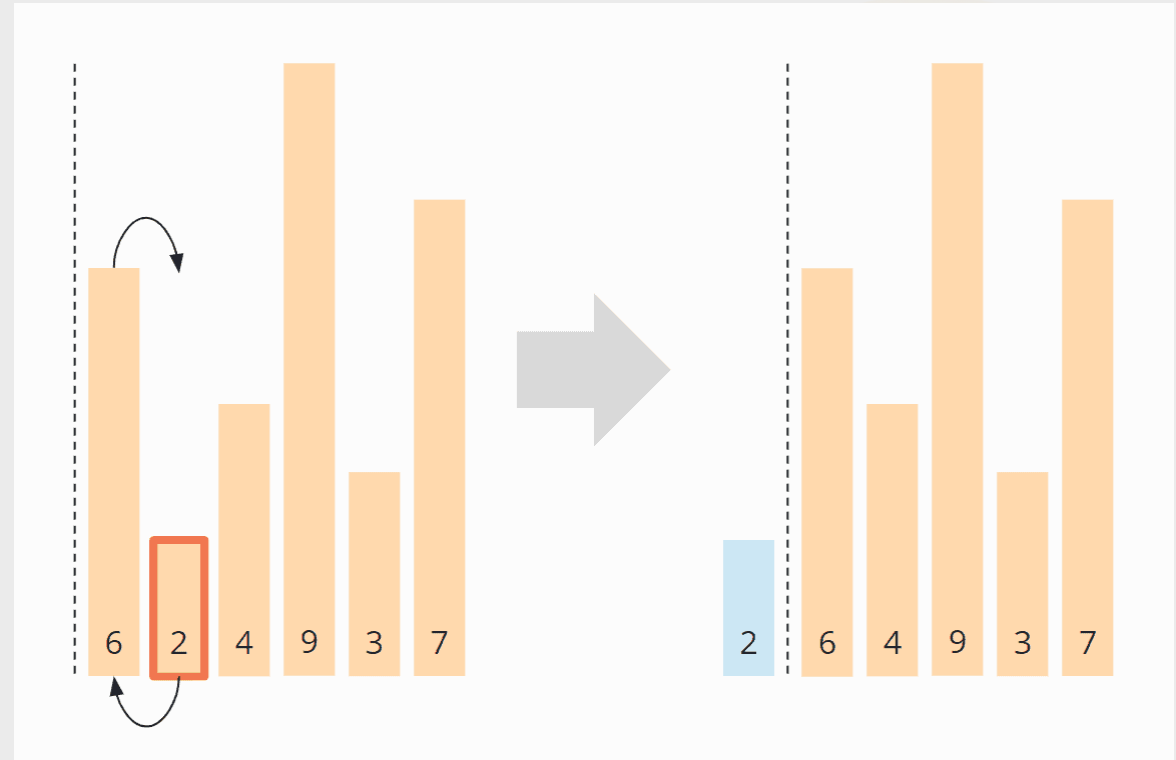
We divide the array into a left, sorted part and a right, unsorted part. The sorted part is empty at the beginning:



Step2

We search for the smallest element in the right, unsorted part. To do this, **we first remember the first element**, which is 6. We go to the next field, where we find an even smaller element is 2. We walk over the rest of the array, looking for a smaller element. Since we can't find one, we stick with 2. We put it in the correct position by swapping it with the element in the first place. Then we move the border between the array sections one field to the right:

We search again in the right, unsorted part for the smallest element. This time it is 3; we swap it with the element in the second position:

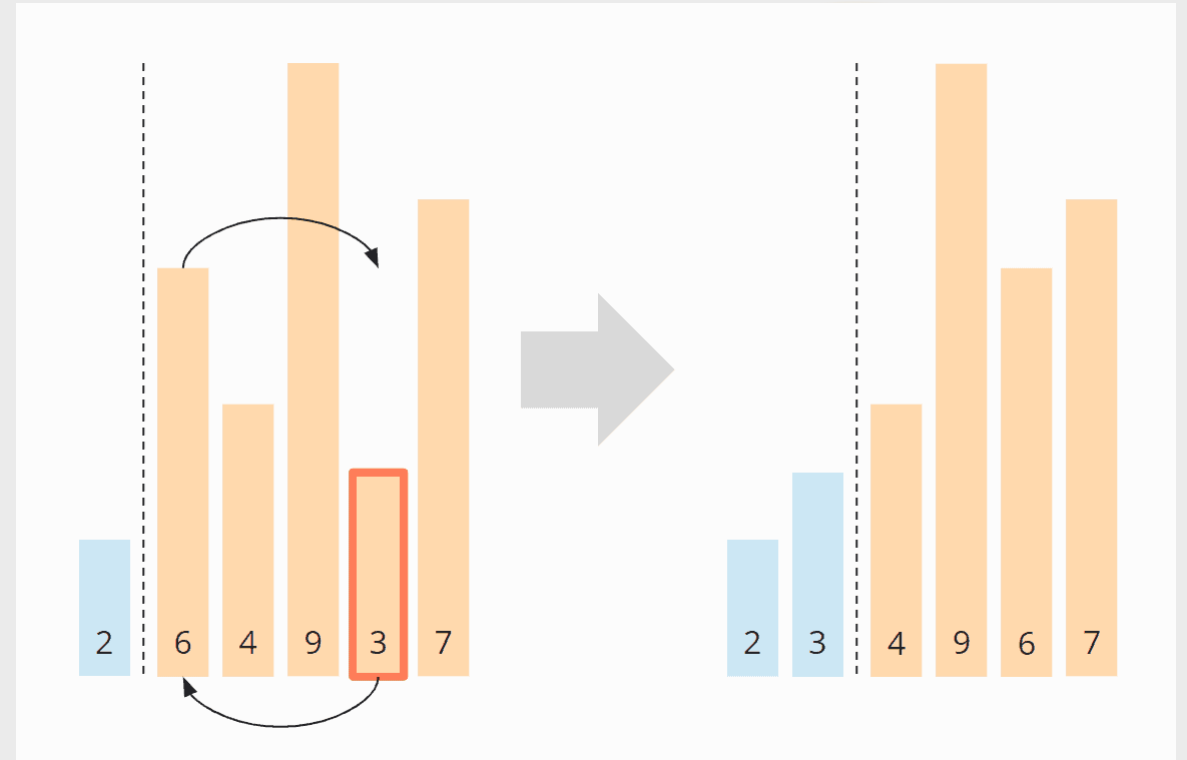


Animation”

<https://afteracademy.com/blog/introduction-to-sorting-algorithms-selection-and-insertion-sort>

Step3

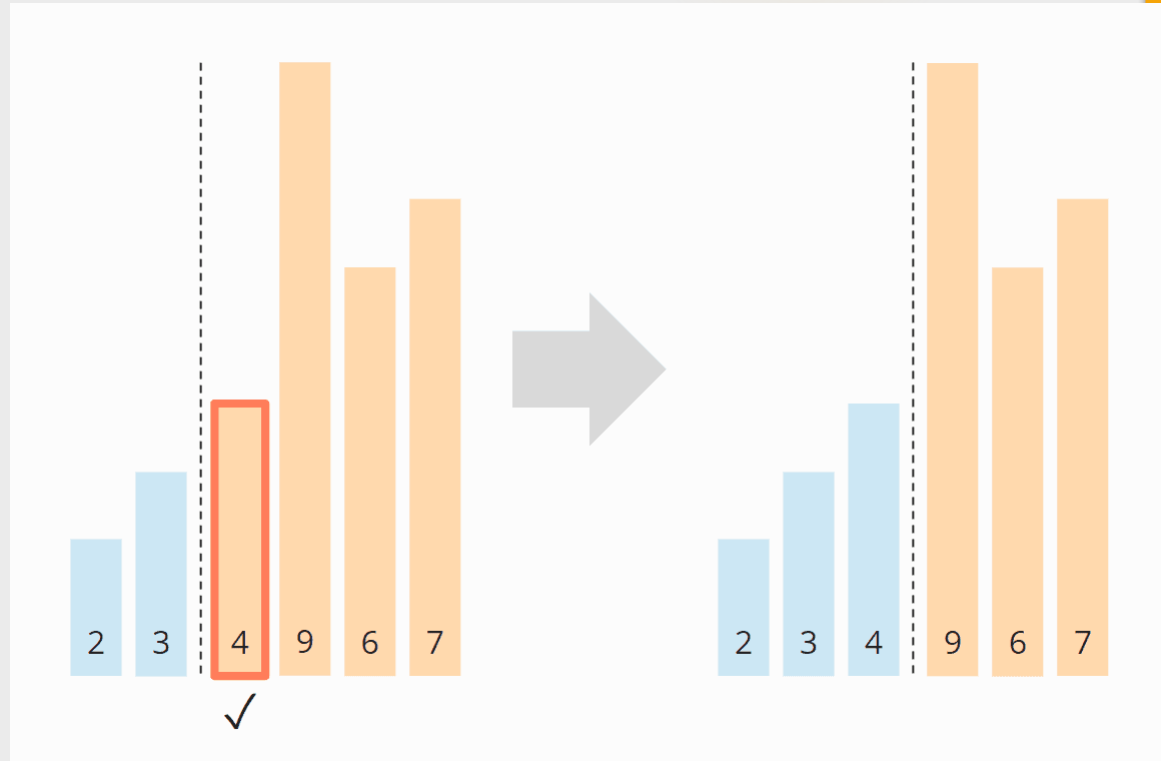
We search again in the right, unsorted part for the smallest element. This time it is the 3; we swap it with the element in the second position:



Step4

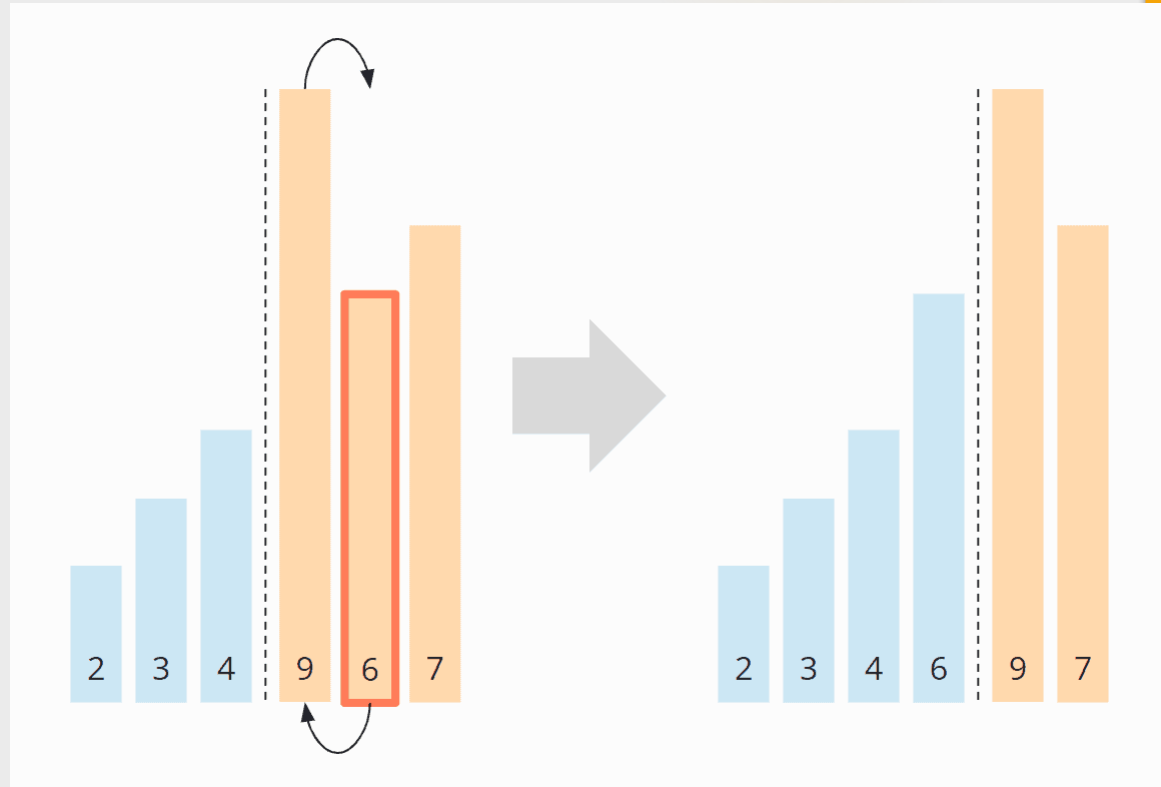
Again we search for the smallest element in the right section.

It is the 4, which is already in the correct position. So there is no need for swapping operation in this step, and we just -move the section border:



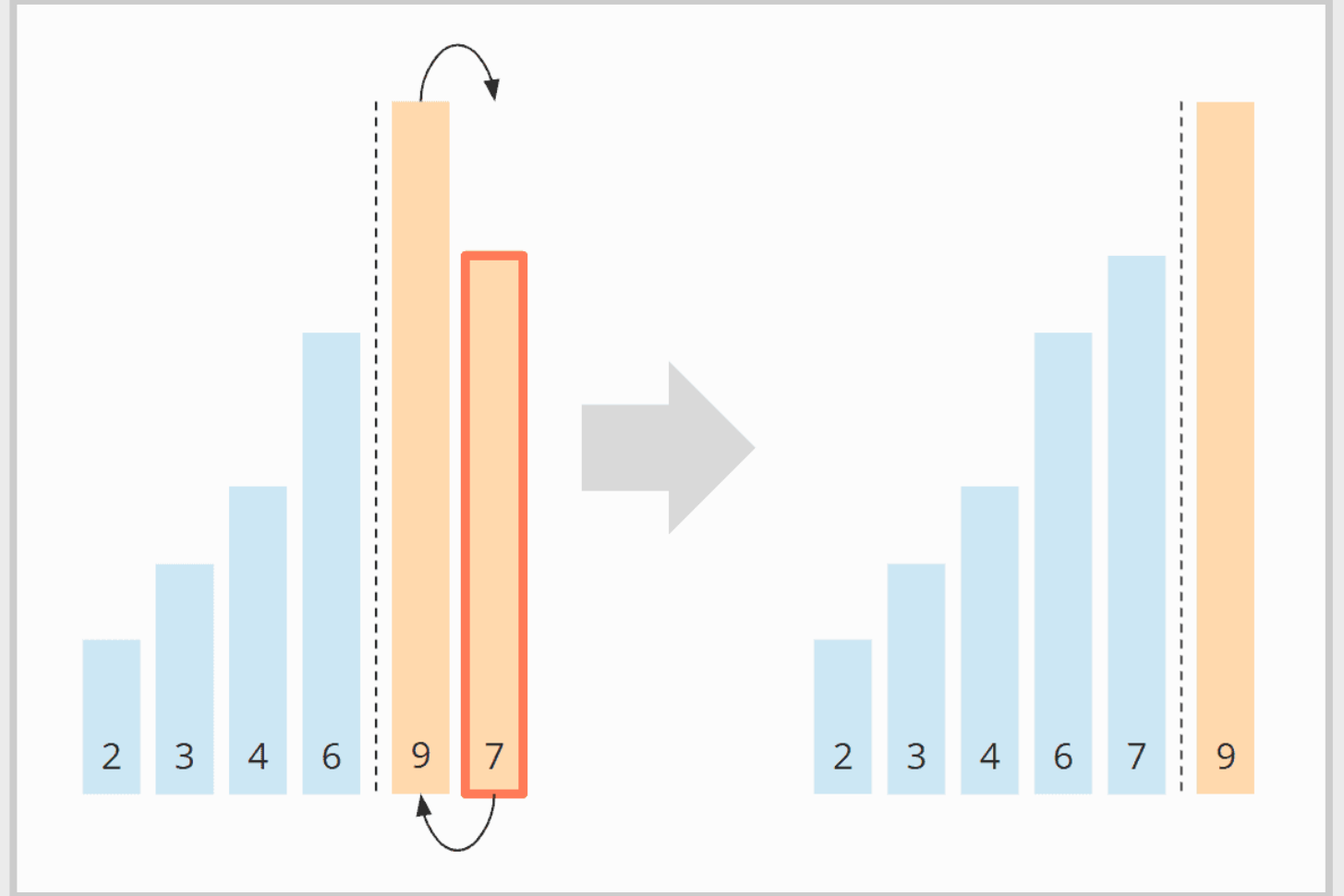
Step5

As the smallest element, we find the 6. We swap it with the element at the beginning of the right part, the 9:



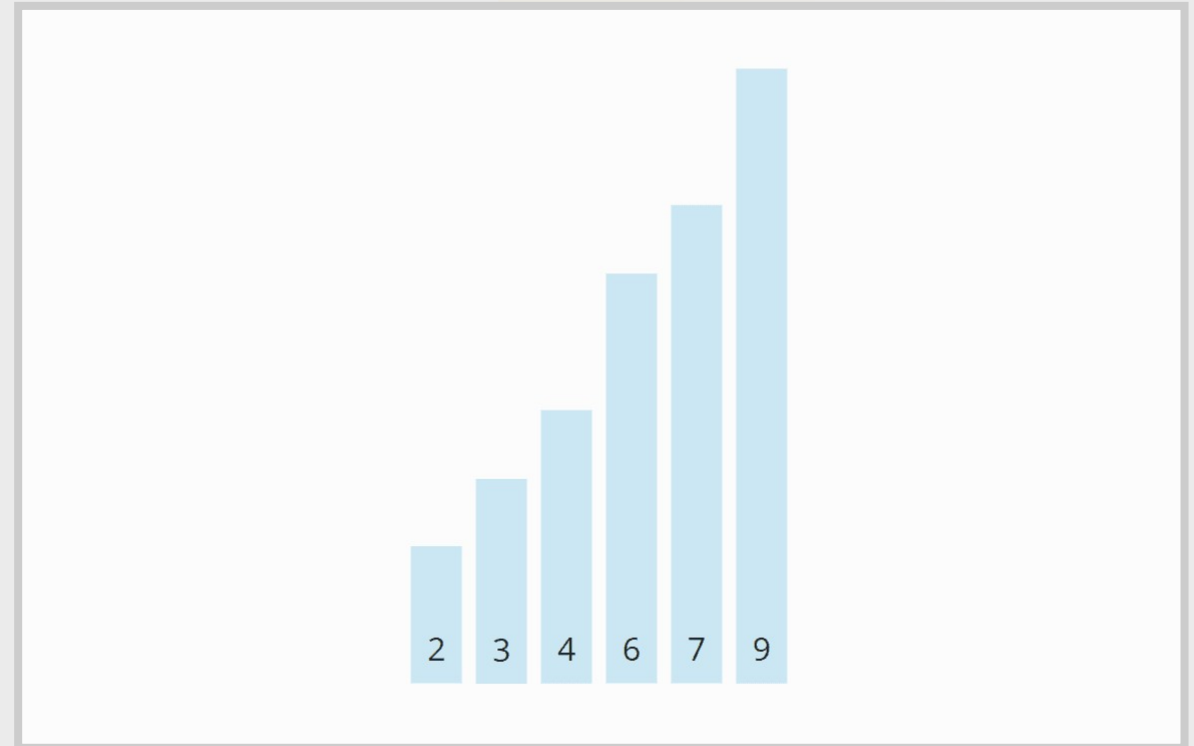
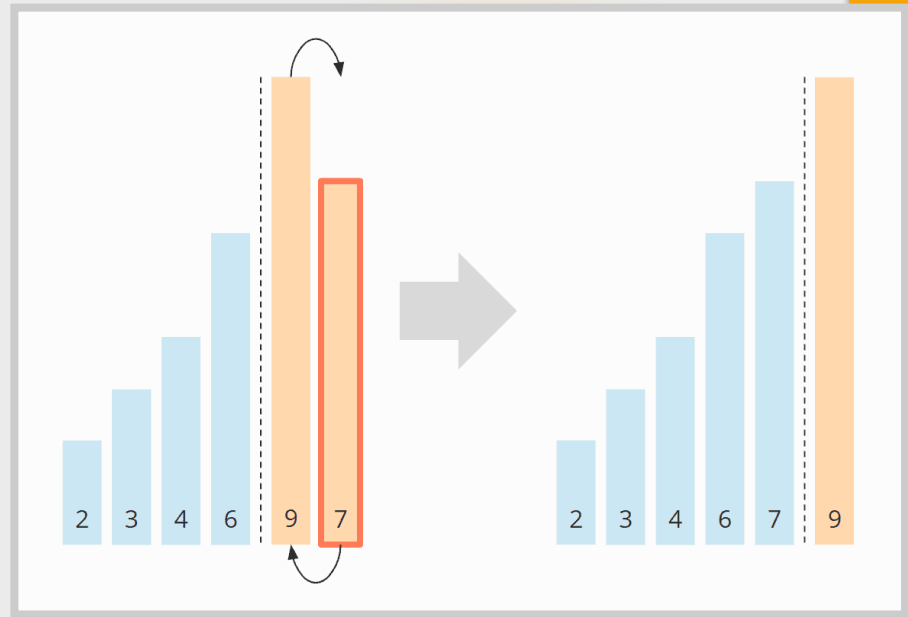
Step 6

Of the remaining two elements, the 7 is the smallest. We swap it with the 9:



Algorithm finished

The last element is automatically the largest and, therefore, in the correct position. The algorithm is finished, and the elements are sorted:



C++ example

```
#include<iostream>
using namespace std;

void selectionSort(int a[],int n){
    for(int i=0;i<n-1;i++){
        int targetIndex= i;
        for(int j = i+1;j<n;j++){
            if(a[j]<a[targetIndex]){
                targetIndex = j;
            }
        }
        swap(a[i],a[targetIndex]);
    }
}
```

```
int main(){
    int n;
    cout<<"Enter the number of elements"<<endl;
    cin>>n;
    int a[n];
    cout<<"Enter "<<n<<" elements"<<endl;
    for(int i=0;i<n;i++){
        cin>>a[i];
    }
    selectionSort(a,n);
    for(int i=0;i<n;i++){
        cout<<a[i]<<" ";
    }
    return 0;
}
```

Selection sort complexity

The selection sort will take the time complexity of $O(n^2)$. The outer loop is taking $n-1$ steps and inner loop $n-2$ steps.

$$(n-1) * (n-1) = n^2 + kn + c$$
$$\approx n^2$$

Merge Sort

Recursive Algorithm

Merge Sort

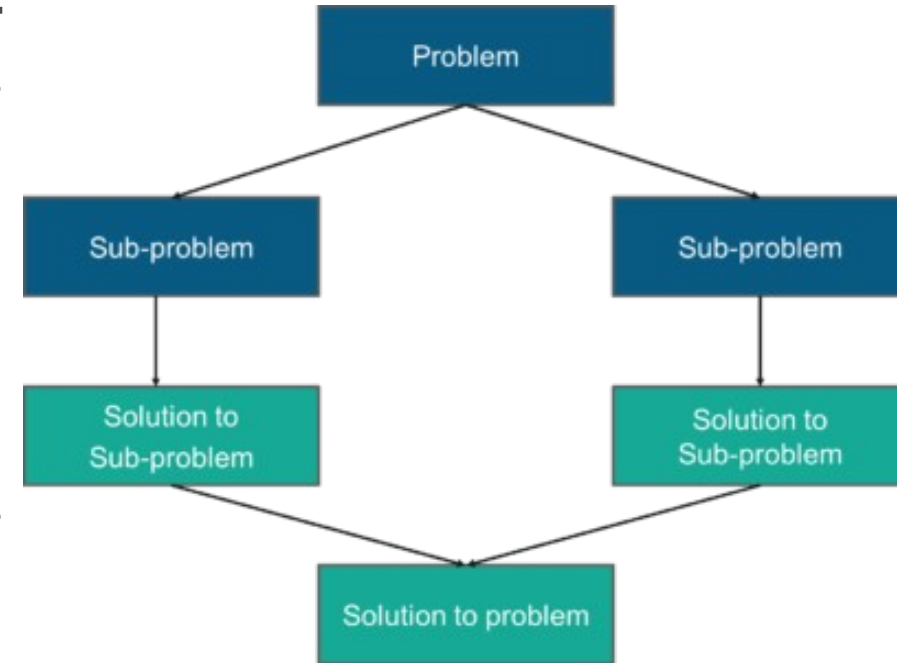
Merge sort is one of the best examples of **Divide & Conquer algorithm**. Before we discuss a lot about Merge sort algorithm, let us understand **Divide & Conquer** technique.

In Divide & Conquer algorithm design paradigm, we divide the problem in sub-problems recursively then solve the sub-problems, & at last combine the solutions to find the final result.

One thing to keep in mind while dividing the problems into sub-problems is that, the structure of sub-problems should not change as of the original problem.

Divide & Conquer algorithm has 3 steps:

1. **Divide**: Breaking the problem into subproblems
 - Compute Midpoint/mid index $(m) = (left + right) / 2$
 - Here left is the starting(low) index & right is the last(high) index of the array(list)
2. **Conquer**: Recursively solving the subproblems
 - Call Merge Sort on the left sub-array (sub-list) starting from left to midpoint
 - Call Merge Sort on the right sub-array (sub-list) starting from midpoint+1 to high index
3. **Combine**: Combining the solutions to get the final result.
 - Call merge function to merge the divided sub-arrays back to the original array.
 - Perform sorting of these smaller sub arrays before merging them back.



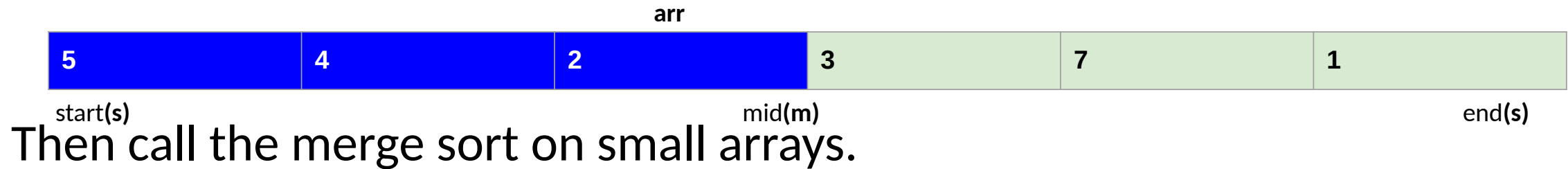
Merge Sort (next...)

In Merge sort, we divide the array recursively in two halves, until each sub-array contains a single element, and then we merge the sub-array in a way that it results into a sorted array.

merge arrays function merges two sorted sub-arrays into one, wherein it assumes that `array[l .. n]` and `arr[n+1 .. r]` are sorted.

Merge sort(next...)

Merge sort is a very fast recursive sorting algorithm. If we want to sort an empty array or with one element, the array is sorted. Then what about the recursive case. We must divide the array in two parts:



After recursive merge sort, the two arrays are sorted

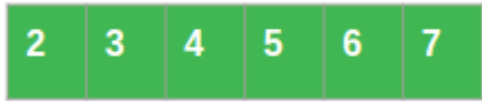


Then merge the two sorted arrays

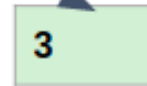
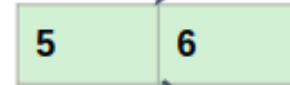
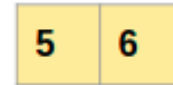
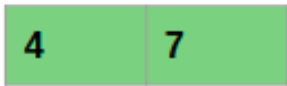
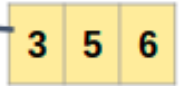
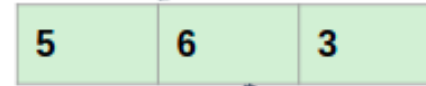
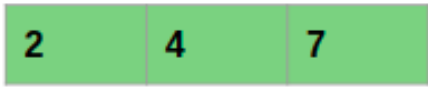


Merge Sort: Let us have an example

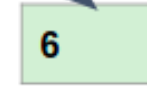
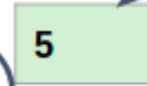
Then merge the 2 sorted arrays



Merge array of 4, 7 and second array with 2



Merge 7 and 4



Merge array of 5, 6 and second array with 3

Note:

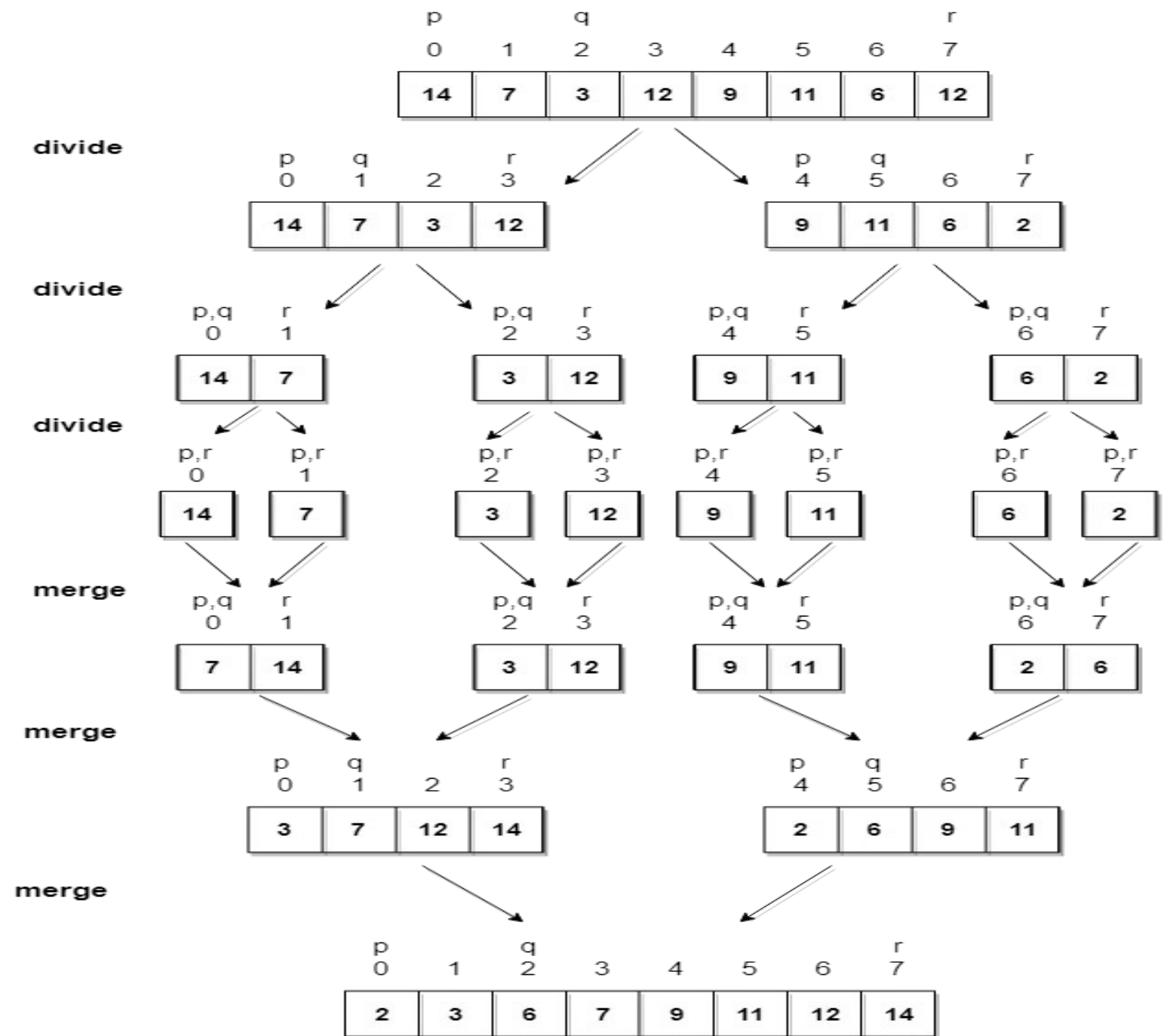
1. The empty array, or the array of 1 element is sorted.
2. Divide the unsorted list into n sublists, each containing one element (a list of one element is considered sorted).
3. Repeatedly merge sublists to produce new sorted sublists until there is only one sublist remaining.

Example 3

Assume we have an array with values

{14, 7, 3, 12, 9, 11, 6, 12}

Observe the visual representation of how merge sort will sort the given array.



Merge Arrays

```
void mergeArrays(int x[],int y[],int a[],int s,int e){
    int mid = (s+e)/2;
    int i=s;
    int j = mid+1;
    int k = s;
    while(i<=mid && j<=e){
//We need to do the Comparison
        if(x[i] < y[j]){
            a[k] = x[i];
            i++;
            k++;
        }else{
            a[k] = y[j];
            j++;
            k++;
        }
    }
    //Copy Rest of Elements
    while(i<=mid){
        a[k] = x[i];
        k++;
        i++;
    }
    while(j<=e){
        a[k] = y[j];
        k++;
        j++;
    }
}
```

Note:

x: Sorted Array

y: Sorted Array

a: Array to be sorted

s: Starting index

mid: Middle Index

e: End Index

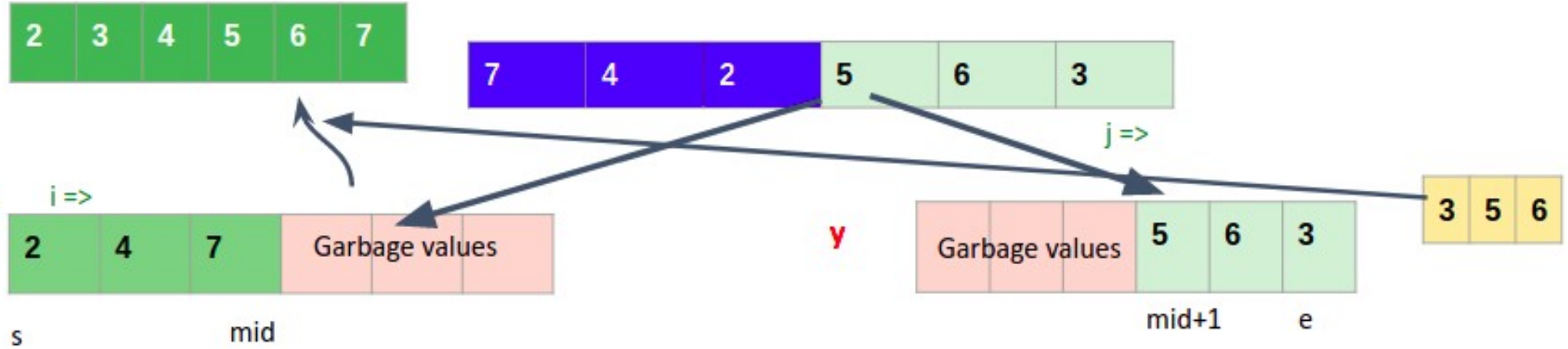
```
void mergeSort(int a[],int s,int e){
    if(s>=e){
        return;
    }
    int mid = (s+e)/2;
    int x[100],y[100];
    for(int i=s;i<=mid;i++){
        x[i] = a[i];
    }
    for(int i=mid+1;i<=e;i++){
        y[i] = a[i];
    }
    mergeSort(x,s,mid);
    mergeSort(y,mid+1,e);
    mergeArrays(x,y,a,s,e);
}
```

Sort Array

```
int main(){
    int a[] = {7,3,10,5,6,2,-5};
    mergeSort(a,0,6);
    for(int i=0;i<7;i++){
        cout<<a[i]<<" ";
    }
    return 0;
}
```

Merging two arrays

Then merge the 2 sorted arrays



Merge array of 4, 7 and second array with 2

6 will be copied to a but not 7.

To come out of the array, there are two options, (1) The value of $i = \text{mid} + 1$ or (2) The value of $j = e + 1$. Then we need to copy the rest of elements.

//Copy Rest of Elements

```
while(i<=mid){
    a[k] = x[i];
    k++;
    i++;
}
while(j<=e){
    a[k] = y[j];
    k++;
    j++;
}
```

Another option with pointers

```
void merge(int *a,int s,int e){
    int mid = (s+e)/2;

    int i = s;
    int j = mid+1;
    int k = s;

    int temp[100];

    while(i<=mid && j<=e){
        if(a[i] < a[j]){
            temp[k++] = a[i++];
        }
        else{
            temp[k++] = a[j++];
        }
    }
    while(i<=mid){
        temp[k++] = a[i++];
    }
    while(j<=e){
        temp[k++] = a[j++];
    }

    //We need to copy all element to original arrays
    for(int i=s;i<=e;i++){
        a[i] = temp[i];
    }

}
```

```
void mergeSort(int a[],int s,int e){
    //Base case - 1 or 0 elements
    if(s>=e){
        return;
    }

    //Follow 3 steps
    //1. Divide
    int mid = (s+e)/2;

    //Recursively the arrays - s,mid and
    mid+1,e
    mergeSort(a,s,mid);
    mergeSort(a,mid+1,e);

    //Merge the two parts
    merge(a,s,e);

}
```

```
int main(){
    int a[] = {7,3,10,5,6,2,-5};
    mergeSort(a,0,6);
    for(int i=0;i<7;i++){
        cout<<a[i]<<" ";
    }
    return 0;
}
```

Merge sort complexity

To analyse the merge sort complexity add the print statement in mergesort() function right at the beginning

As we have 7 elements in the array, the program will print **6** one's.

Comparing to 7 elements we have, the $\log(7)=0.845$

$$n * \log(n) = 7 * 0.845 = 5.91 \\ \approx 6$$

```
void mergeSort(int a[],int s,int e){  
    //Base case - 1 or 0 elements  
    if(s>=e){  
        return;  
    }  
    cout<<1<<endl;  
    //Follow 3 steps  
    //1. Divide  
    int mid = (s+e)/2;  
  
    //Recursively the arrays - s,mid and  
    mid+1,e  
    mergeSort(a,s,mid);  
    mergeSort(a,mid+1,e);  
  
    //Merge the two parts  
    merge(a,s,e);  
}
```

Complexity Analysis of Merge Sort

Merge Sort is quite fast, and has a time complexity of $O(n \log n)$. It is also a stable sort, which means the "equal" elements are ordered in the same order in the sorted list.

In this section we will understand why the running time for merge sort is $O(n \log n)$.

Whenever we divide a number into half in every step, it can be represented using a logarithmic function, which is $\log n$ and the number of steps can be represented by $\log n + 1$ (at most)

Also, we perform a single step operation to find out the middle of any subarray, i.e. $O(1)$.

And to merge the subarrays, made by dividing the original array of n elements, a running time of $O(n)$ will be required.

Hence the total time for `mergeSort` function will become $n(\log n + 1)$, which gives us a time complexity of $O(n \log n)$.

Worst Case Time Complexity [Big-O]: $O(n \log n)$

Best Case Time Complexity [Big-omega]: $O(n \log n)$

Average Time Complexity [Big-theta]: $O(n \log n)$

Space Complexity: $O(n)$

- Time complexity of Merge Sort is $O(n \log n)$ in all the 3 cases (worst, average and best) as merge sort always divides the array in two halves and takes linear time to merge two halves.
- It requires equal amount of additional space as the unsorted array. Hence it's not at all recommended for searching large unsorted arrays.
- It is the best Sorting technique used for sorting **Linked list**.

Quick Sort Algorithm

Recursive Algorithm

Quick Sort

Quick Sort is also a recursive algorithm which is very fast sorting algorithm. The key process in quickSort is a partition().

What is the Quick Sort?

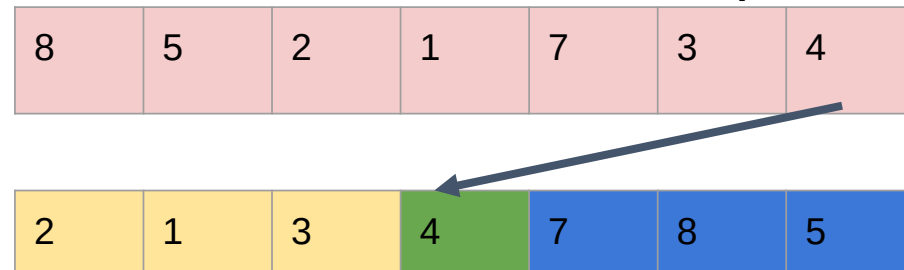
Consider an array $a=\{8,5,2,1,7,3,4\}$

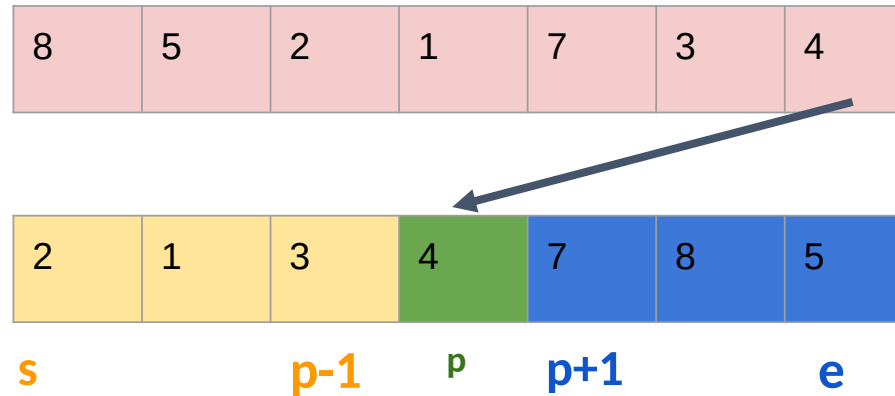
The target of partitions is, given an array and an element x of an array as the **Pivot** and put x at its correct position in a sorted array. All the elements greater than the **Pivot** will be present at the right and all elements less than the **Pivot** will be present at the left.

In our example of array a , the Pivot is 4;

The **Pivot** is 4. Now we **put 4(Pivot) in its correct position** and next we rearrange all elements based on the Pivot. All the elements greater than the 4 will be present at the right and all elements less than 4 will be present at the left even if they are not sorted.

This process is Partition=>





After partitioning, we will apply Quick Sort on the left array and Quick Sort on the right array. After applying Quick Sort on the left array it will be sorted(1,2,3). Then after applying Quick Sort on the right array, it will be Sorted(5,7,8).



Then the array is Sorted.

How to calculate the Index of the Pivot for the Sorted Array ?

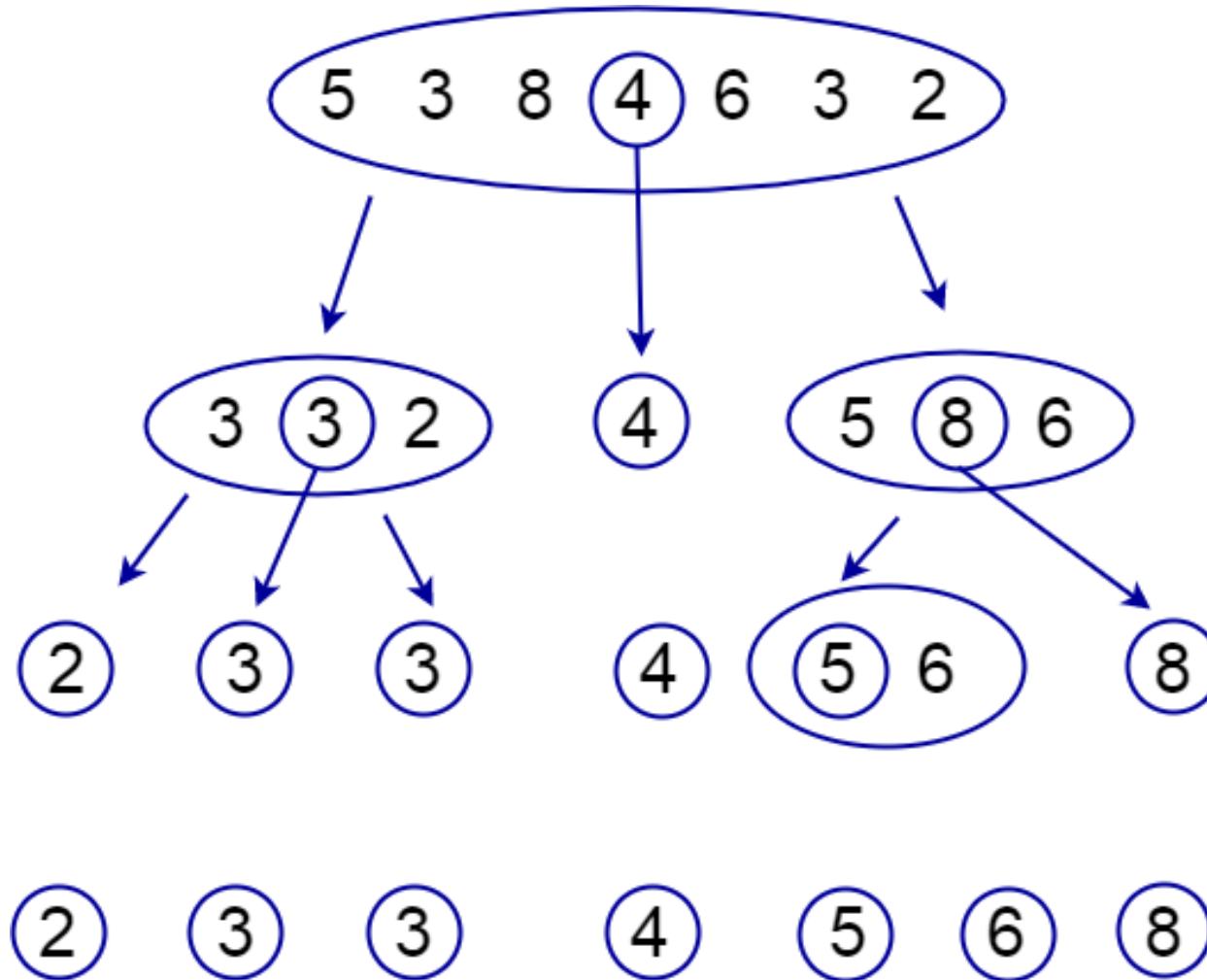
To know the index of the Pivot, we need to count the elements which are less than the Pivot. The resulting value is the correct index for the Pivot. Then swap the element there and the Pivot.

Note: The Pivot can be any element that is selected by the user as the starting element.

There are many different versions of quickSort that pick pivot in different ways.

- Always pick the first element as a pivot.
- Always pick the last element as a pivot.
- Pick a random element as a pivot.
- Pick median as the pivot.

Graphical Representation



Pseudo Code

QuickSort function:

```
/* low -> Starting index, high -> Ending index */  
quickSort(arr[], low, high) {  
    if (low < high) {  
        /* pi is partitioning index, arr[pi] is now at right place */  
        pi = partition(arr, low, high);  
        quickSort(arr, low, pi - 1); // Before pi  
        quickSort(arr, pi + 1, high); // After pi  
    }  
}
```

Partition function:

```
partition (arr[], low, high)  
{  
    // pivot (Element to be placed at right position)  
    pivot = arr[high];  
    i = (low - 1) // Index of smaller element and indicates the  
    // right position of pivot found so far  
    for (j = low; j <= high- 1; j++){  
        // If current element is smaller than the pivot  
        if (arr[j] < pivot){  
            i++; // increment index of smaller element  
            swap arr[i] and arr[j]  
        } }  
    swap arr[i + 1] and arr[high])  
    return (i + 1)  
}
```

Quick Sort with C++

Pivot as the Last element

```
int partition(int a[],int s,int e){  
    int i=s;  
    int pivot = a[e];  
    for(int j=s;j<=e-1;j++){  
        if(a[j] < pivot){  
            swap(a[i],a[j]);  
            i++;  
        }  
    }  
    swap(a[i],a[e]);  
    return i;  
}
```

```
void quickSort(int a[],int s,int e){  
    if(s>=e){  
        return;  
    }  
    int p = partition(a,s,e);  
    quickSort(a,s,p-1);  
    quickSort(a,p+1,e);  
}
```

```
int main(){  
  
    int a[] = {2,1,3,4,5,7,-2};  
    quickSort(a,0,6);  
    for(int i=0;i<7;i++){  
        cout<<a[i]<<" ";  
    }  
  
    return 0;  
}
```

Heap Sort